

Why C++ Templates Must Be Defined in Headers ?

6 min read · Jan 22, 2026



Sagar

Following ▾



Listen



Share

... More



C++ templates live in headers for a reason — because compile-time power demands full visibility, and that trade-off is what gives templates their unmatched flexibility and performance. But you must be wondering why in header ? This post is going to answer this question **Why C++ Templates Must reside in header ?**

Lets Understand the problem first

What happens when you put the template declaration and its definition in separate .h and .cpp files. Let's start with a real-world example that is going to cause a linker error:

```
// vector.h
template<typename T>
class vector {
public:
    T& at(int);
};
```

```
// vector.cpp
#include "vector.h"

template <typename T>
T& vector<T>::at(int i) {
    // some code
}
```

```
// main.cpp
#include "vector.h"

int main() {
    vector<int> a;
    a.at(5);
}
```

When you compile this:

```
g++ vector.cpp main.cpp
```

You get a linker error:

```
/usr/bin/ld: /tmp/cc6PAyEd.o: in function `main':
main.cpp:(.text+0x28): undefined reference to `vector<int>::at(int)'
```

```
collect2: error: ld returned 1 exit status
```

Why does this happen? The answer lies in how C++ compiles templates.

Understanding Two-Phase Translation

C++ templates use a **two-phase translation model**. Understanding this is crucial to understanding why templates must be in headers.

Phase 1: Template Parsing (Definition Point)

When the compiler first encounters a template definition, it performs **Phase 1** processing:

Phase 1: Template Parsing

- ✓ Parse the syntax
- ✓ Check **template** structure
- ✓ Store the **template** body
- ✓ Resolve non-dependent names
- ✗ Does NOT generate actual code
- ✗ Does NOT substitute **template** parameters

In our example (vector.cpp):

```
template <typename T>
T& vector<T>::at(int i) {
    // some code
}
```

During Phase 1:

- The compiler parses this template

- Checks that the syntax is valid
- Stores the template definition internally
- **No actual code is generated yet**
- The compiler doesn't know what τ will be

Phase 2: Template Instantiation (Usage Point)

Phase 2 happens when you **use** the template with a specific type:

Phase 2: Template Instantiation

- ✓ Triggered when template **is** USED
- ✓ Substitute template parameters ($T = \text{int}$)
- ✓ Resolve **all type**-dependent operations
- ✓ Check **if** operations are valid **for type**
- ✓ Generate actual machine code

In our example (main.cpp):

```
vector<int> a;  
a.at(5);
```

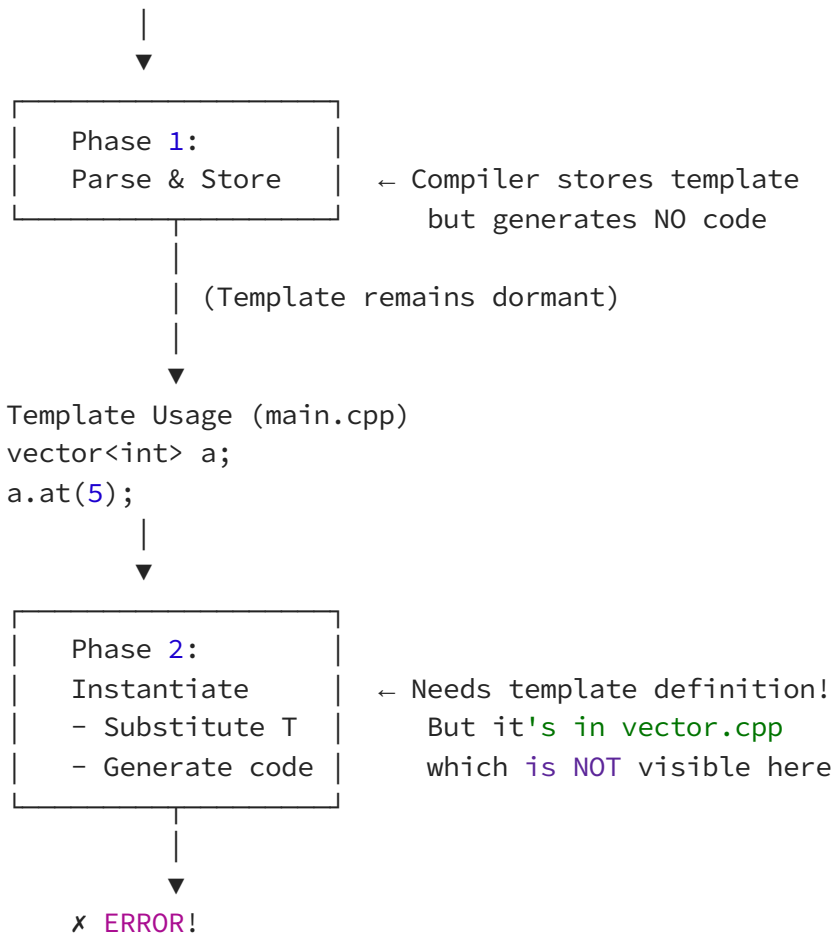
When the compiler sees `a.at(5)`, it needs to:

1. Substitute $\tau = \text{int}$
2. Generate the actual code for `vector<int>::at(int)`
3. Check if all operations are valid for `int`

Critical Point: To do Phase 2, the compiler **must see the complete template definition!**

The Complete Flow:

Template Definition (vector.cpp)



Why the Linker Error Occurs ?

Let's trace exactly what happens during compilation:

Step 1: Compile vector.cpp

```
g++ -c vector.cpp -o vector.o
```

Compiling vector.cpp

- Compiler sees **template** definition
- Phase 1: Parses and stores **template**
- Phase 2: NOT TRIGGERED

- ```
(template is never used in this file)
```
- Result: `vector.o` contains NO code  
`for vector<int>::at(int)`

Even though `vector.cpp` has the template definition, no `vector<int>::at(int)` code is generated because the template is never instantiated in this file.



## Step 2: Compile `main.cpp`

```
g++ -c main.cpp -o main.o
```

Compiling `main.cpp`

- `#include "vector.h"` → only declaration
- Compiler sees: `a.at(5)`
- Needs to instantiate `vector<int>::at(int)`
- Phase 2 TRIGGERED
- Problem: Can only see declaration  
NOT the definition!
- Cannot generate code without definition
- Assumes definition exists elsewhere
- Creates undefined reference

`main.cpp` only sees the declaration from `vector.h`:

```
template<typename T>
class vector {
public:
```

```
T& at(int); // ← Only this is visible
};
```

It **cannot** see the definition in `vector.cpp`:

```
template <typename T>
T& vector<T>::at(int i) { // ← This is INVISIBLE
 // some code
}
```

### Step 3: Linking

```
g++ vector.o main.o -o program
```

#### Linking

- Linker examines `main.o`
- Finds: needs `vector<int>::at(int)`
- Searches `vector.o` for this symbol
- `vector.o` has NO such symbol (wasn't instantiated there)
- ✗ LINKER ERROR  
"undefined reference to  
`vector<int>::at(int)`"

Visualization of the Problem:

`vector.cpp`

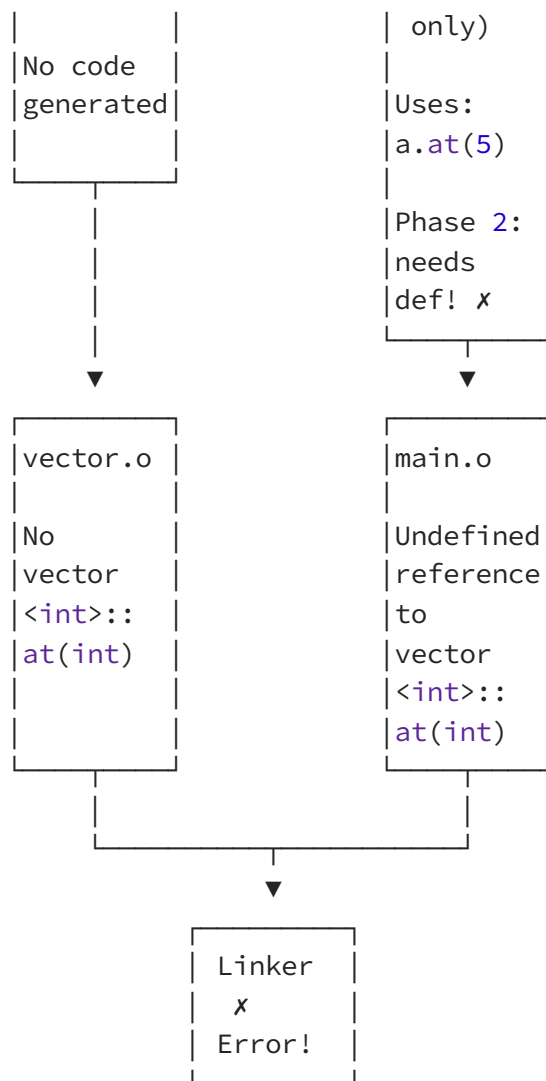


```
Phase 1:
Parse
template
```

`main.cpp`



```
Includes
vector.h
(decl
```



### Why Separate Compilation is the Issue ?

Each .cpp file is a **separate translation unit**:

Project Structure:

```
├─ vector.cpp → Compiled independently → vector.o
├─ main.cpp → Compiled independently → main.o
└─ vector.h → Included in both files
```

- vector.cpp and main.cpp **don't see each other** during compilation
- They are compiled completely separately
- The compiler cannot "look ahead" to see what's in other files
- When compiling main.cpp, the compiler has **no idea** that vector.cpp exists



## The Solution

The solution is simple: **Put the template definition in the header file.**

*Understanding the Key Insight:*

**Templates don't emit code until instantiated, so include the .cpp in the .h instead of the other way around!**

This is the opposite of what you do with regular C++ code:

- **Regular code:** Declare in .h , define in .cpp , include the .h
- **Template code:** Declare in .h , define in .cpp , **include the .cpp at the end of the .h**

Or better yet: Just put everything in the .h file!

Corrected Code will be like this:

```
// vector.h
template<typename T>
class vector {
public:
 T& at(int i) {
 // some code - DEFINITION IN HEADER
 }
};
```

Or if you prefer to separate declaration and definition:

```
// vector.h
template<typename T>
class vector {
public:
 T& at(int);
};

// Definition still in the header
template<typename T>
```

```
T& vector<T>::at(int i) {
 // some code
}
```

```
// main.cpp
#include "vector.h"

int main() {
 vector<int> a;
 a.at(5); // ✓ Works!
}
```

## Why This Works ?

Compiling main.cpp

- `#include "vector.h"`  
→ Full definition is included
- Compiler sees: `a.at(5)`
- Phase 2: Instantiate `vector<int>::at(int)`
- Template definition IS VISIBLE
- Compiler substitutes `T = int`
- Generates actual code
- Code is placed in `main.o`
- ✓ SUCCESS - No linker error

Now delete `vector.cpp` entirely - you don't need it!

```
g++ main.cpp -o program # ✓ Works!
```

*Key Principle:*

**Template definitions must be visible at the point of instantiation.**

Since instantiation happens wherever the template is **used** (not where it's **defined**), the definition must be in a header file that can be included everywhere.

## Alternative Solutions

While putting definitions in headers is the standard approach, there are alternatives:

### Solution 1: Explicit Template Instantiation

If you know **exactly** which types will be used, you can explicitly instantiate them in the `.cpp` file:

```
// vector.h
template<typename T>
class vector {
public:
 T& at(int);
};
```

```
// vector.cpp
#include "vector.h"

template <typename T>
T& vector<T>::at(int i) {
 // some code
}

// Explicit instantiation for specific types
template class vector<int>; // Generate vector<int>
template class vector<double>; // Generate vector<double>
template class vector<std::string>; // Generate vector<string>
```

```
// main.cpp
#include "vector.h"

int main() {
 vector<int> a;
 a.at(5); // ✓ Works! vector<int> was explicitly instantiated
}
```

**Compilation:**

```
g++ vector.cpp main.cpp # ✓ Works!
```

### Pros:

- Definitions can stay in .cpp files
- Faster compilation for large projects
- Reduces code bloat

### Cons:

- Must know all types in advance
- Users cannot use the template with new types
- Less flexible - not truly generic

### Solution 2: Include Implementation at End of Header

```
// vector.h
template<typename T>
class vector {
public:
 T& at(int);
};

#include "vector.cpp" // or "vector_impl.h"
```

```
// vector.cpp (or vector_impl.h)
template <typename T>
T& vector<T>::at(int i) {
 // some code
}
```

### Pros:

- Separates interface from implementation (for readability)

- Still makes definition visible

### Cons:

- Confusing naming conventions
- More files to manage

### Summary: The Golden Rule

*Template code must be visible where it's instantiated, not where it's defined.*

Since instantiation happens at the **point of use**, and `.cpp` files are compiled separately, template definitions must be in **headers** that can be included wherever needed.

Found this article helpful? Please **Clap** 🙌 and **follow** for more C++ and system programming content.

Cplusplus

Cpp Templates

Template Metaprogramming

Programming

Software Development



Following ▾

## Written by Sagar

135 followers · 5 following

System Programmer | C++ | Linux System Programming | Rust | Python | Cloud Computing | DevOps | Low Latency Backends